جزوه جامع و بی‌نظیر فصل دوم (بخش ۹ و ۱۰: ساخت، راه‌اندازی و عیب‌یابی)

این جزوه بر اساس بخش‌های نهم و دهم از فصل دوم فایل ارسالی شما و با هدف پوشش **کامل، عمیق و مفهومی** تمام مطالب، تعاریف فنی، فرآیندها و مثال‌های ذکر شده تهیه شده است. هدف، درک کامل مفاهیم برای کسب بالاترین نمره است.

۹-۲: ساخت و راه‌اندازی یک سیستم‌عامل (System Generation & Booting)

پس از طراحی و پیاده‌سازی، یک سیستم‌عامل باید برای یک سخت‌افزار خاص پیکربندی، کامپایل و نصب شود. در این بخش، با فرآیند "جان بخشیدن" به یک سیستم‌عامل آشنا می‌شویم.

۹-۲-۱: تولید سیستم‌عامل (System Generation - SYSGEN)

یک نسخه از سیستم‌عامل معمولاً برای کلاس وسیعی از سخت‌افزارها طراحی می‌شود. فرآیند SYSGEN، آن نسخه عمومی را برای یک پیکربندی سخت‌افزاری **خاص** آماده و بهینه‌سازی می‌کند.

- **مراحل** کلی ساخت یک سیستم لینوکس از **ابتدا:**
  1. **دریافت کد منبع:** دانلود آخرین نسخه از کد منبع هسته لینوکس.
  2. **پیکربندی (Configuration):** این مهم‌ترین مرحله است. با استفاده از ابزارهایی مانند `make menuconfig`، مدیر سیستم مشخص می‌کند که کدام قابلیت‌ها، درایورها و ماژول‌ها باید در هسته گنجانده شوند. این تنظیمات در یک فایل به نام `.config` ذخیره می‌شوند.
  3. **کامپایل هسته (Compile):** با دستور `make`، کد منبع بر اساس فایل `.config` کامپایل شده و یک **تصویر هسته (Kernel Image)** قابل بوت (مانند `vmlinuz`) تولید می‌شود.
  4. **کامپایل ماژول‌ها:** با دستور `make modules`، ماژول‌هایی که قرار است به صورت پویا بارگذاری شوند، کامپایل می‌شوند.
  5. **نصب (Installation):** با دستوراتی مانند `make modules_install` و `make install`، تصویر هسته و ماژول‌های جدید روی سیستم نصب می‌شوند.
- **رویکردهای مختلف SYSGEN:**
  - **کامپایل کامل:** تمام کد بر اساس فایل پیکربندی کامپایل می‌شود. نتیجه یک هسته کاملاً بهینه برای سخت‌افزار مورد نظر است.
  - **انتخاب ماژول‌های از پیش‌کامپایل‌شده:** سیستم‌عامل مجموعه‌ای از ماژول‌های آماده را فراهم می‌کند و در زمان نصب، فقط ماژول‌های لازم بر اساس سخت‌افزار شناسایی شده، به هم پیوند داده می‌شوند. این روش سریع‌تر است اما ممکن است به اندازه روش اول بهینه نباشد.
  - **سیستم کاملاً ماژولار:** انتخاب‌ها در زمان اجرا صورت می‌گیرد و سیستم حداکثر انعطاف‌پذیری را دارد.

۹-۲-۲: راه‌اندازی سیستم (System Boot)

بوت کردن، فرآیند شروع به کار کامپیوتر از لحظه فشردن دکمه پاور تا بارگذاری کامل سیستم‌عامل است.

- **مراحل فرآیند بوت:**

1. **اجرای بارگذار بوت اولیه:** با روشن شدن سیستم، یک برنامه کوچک به نام **برنامه بالاآورنده (Bootstrap Program)** از سفت‌افزار (Firmware) نامیرایی مانند **BIOS** یا **UEFI** اجرا می‌شود.
2. **اجرای بارگذار بوت ثانویه:** بارگذار بوت اولیه، یک بارگذار بوت قدرتمندتر (مانند **GRUB** در لینوکس) را از یک مکان مشخص روی دیسک به نام **بلاک بوت (Boot Block)** پیدا کرده و اجرا می‌کند.
3. **بارگذاری هسته:** GRUB تصویر فشرده هسته (`vmlinuz`) را در حافظه بارگذاری و آن را از حالت فشرده خارج می‌کند.
4. **ایجاد سیستم فایل RAM موقت (initramfs):** GRUB یک سیستم فایل موقت در RAM ایجاد می‌کند که شامل درایورها و ماژول‌های ضروری (مانند درایور کنترلر دیسک) برای دسترسی به سیستم فایل اصلی است.
5. **مقداردهی اولیه و سوار کردن سیستم فایل ریشه:** هسته شروع به اجرا کرده، سخت‌افزار را مقداردهی اولیه می‌کند، درایورهای لازم را از `initramfs` بارگذاری کرده و سپس **سیستم فایل ریشه (Root File System)** اصلی را **سوار (Mount)** می‌کند.
6. **اجرای پردازه init:** هسته اولین پردازه فضای کاربر (`systemd` در لینوکس مدرن) را اجرا می‌کند که مسئول راه‌اندازی سایر سرویس‌ها و آماده‌سازی محیط کاربری است.

- **BIOS در مقابل UEFI:**
  - **BIOS (Basic Input/Output System):** سفت‌افزار قدیمی که از یک فرآیند بوت چند مرحله‌ای استفاده می‌کند.
  - **UEFI (Unified Extensible Firmware Interface):** سفت‌افزار مدرن که سریع‌تر است، از دیسک‌های بزرگتر و سیستم‌های ۶۴ بیتی بهتر پشتیبانی می‌کند و خود یک مدیر بوت کامل است.
- **بوت در موبایل:** فرآیند مشابه است، اما اغلب از بارگذارهای بوت متفاوتی (مانند **LK - Little Kernel** در اندروید) استفاده می‌شود و `initramfs` به عنوان سیستم فایل ریشه دائمی باقی می‌ماند.

۲-۱۰: عیب‌یابی سیستم‌عامل (Operating-System Debugging)

عیب‌یابی (Debugging)، هنر یافتن و رفع خطاها در نرم‌افزار و سخت‌افزار است. عیب‌یابی هسته سیستم‌عامل به دلیل پیچیدگی و دسترسی سطح پایین آن به سخت‌افزار، چالش‌برانگیزتر از عیب‌یابی برنامه‌های عادی است.

۲-۱۰-۱: *تحلیل خرابی (Failure Analysis)*

- **خطای برنامه کاربر:** وقتی یک برنامه کاربر با خطا مواجه می‌شود، سیستم‌عامل آن را خاتمه داده، یک پیغام خطا ثبت می‌کند و ممکن است یک **رونوشت حافظه (Core Dump)** از فضای آدرس آن برنامه تهیه کند تا توسط یک **دیباگر (Debugger)** تحلیل شود.
- **شکست هسته (Kernel Crash):** یک خطا در خود هسته، یک **شکست** نامیده می‌شود. این خطا بسیار جدی‌تر است و معمولاً کل سیستم را متوقف می‌کند.
  - **رونوشت شکست (Crash Dump):** از آنجایی که در هنگام شکست هسته، سیستم فایل ممکن است در وضعیت ناپایداری باشد، سیستم‌عامل نمی‌تواند مستقیماً رونوشت حافظه را در یک فایل بنویسد. در عوض، محتویات کامل حافظه (یا حداقل بخش‌های مربوط به هسته) را در یک **ناحیه** از پیش تعیین‌شده روی **دیسک (Swap Space)** می‌نویسد.
  - **تحلیل:** پس از راه‌اندازی مجدد سیستم، یک برنامه سیستمی این داده‌ها را از آن ناحیه خوانده و آن‌ها را به یک فایل قابل تحلیل تبدیل می‌کند.

۲-۱۰-۲ : کنترل و تنظیم کارایی *(Performance Monitoring and Tuning)*

تنظیم کارایی با هدف شناسایی و رفع **گلوگاه‌ها (Bottlenecks)** در سیستم انجام می‌شود. برای این کار، به ابزارهایی برای نظارت بر رفتار سیستم نیاز داریم.

- **شمارنده‌ها (Counters):** هسته به طور مداوم آمارهای مختلفی را در شمارنده‌ها ثبت می‌کند (مانند تعداد فراخوانی‌های سیستمی، میزان استفاده از CPU، تعداد عملیات I/O). ابزارهای نظارتی این شمارنده‌ها را می‌خوانند و به کاربر نمایش می‌دهند.
  - **ابزارهای لینوکس:**
    - **در سطح پردازه:** `ps`, `top`
    - **در سطح سیستم:** `vmstat` (آمار حافظه)، `netstat` (آمار شبکه)، `iostat` (آمار I/O دیسک).
  - **سیستم فایل `/proc/`:** در لینوکس، این آمارها از طریق یک سیستم فایل مجازی به نام `/proc` که فقط در حافظه وجود دارد، در دسترس قرار می‌گیرند.
- **ردیابی (Tracing):** برخلاف شمارنده‌ها که فقط مقادیر تجمعی را نشان می‌دهند، ابزارهای ردیابی داده‌های دقیقی را برای **رویدادهای خاص** جمع‌آوری می‌کنند.
  - **ابزارهای لینوکس:**
    - **`strace`:** فراخوانی‌های سیستمی انجام شده توسط یک پردازه را ردیابی می‌کند.
    - **`gdb`:** یک دیباگر قدرتمند در سطح کد منبع.
    - **`perf`:** مجموعه‌ای از ابزارهای پیشرفته برای تحلیل کارایی.
    - **`tcpdump`:** بسته‌های شبکه را جمع‌آوری و تحلیل می‌کند.

۲-۱۰-۳ : ابزارهای ردیابی پویا *(BCC)*

BCC (BPF Compiler Collection) یک مجموعه ابزار مدرن و قدرتمند برای ردیابی پویای هسته لینوکس است.

- **eBPF** (extended Berkeley Packet Filter): فناوری زیربنایی BCC که به برنامه‌های کوچک و امن اجازه می‌دهد تا در نقاط مختلفی از هسته (مانند زمان ورود یا خروج از یک فراخوانی سیستمی) **تزریق** شده و داده‌های دقیقی را بدون کاهش قابل توجه کارایی سیستم، جمع‌آوری کنند.
- **ایمنی:** برنامه‌های eBPF قبل از اجرا توسط یک **وارسی‌کننده (Verifier)** در هسته بررسی می‌شوند تا اطمینان حاصل شود که به سیستم آسیب نمی‌رسانند.
- **BCC:** یک رابط کاربری ساده‌تر (معمولاً با زبان پایتون) برای نوشتن و استفاده از ابزارهای مبتنی بر eBPF فراهم می‌کند.
- **مثال:** ابزار `disksnoop.py` از مجموعه BCC، تمام عملیات I/O دیسک را به صورت زنده نمایش می‌دهد و اطلاعاتی مانند زمان، نوع عملیات (خواندن/نوشتن)، حجم داده و **تأخیر (Latency)** را گزارش می‌کند.
- **قدرت BCC:** این ابزارها می‌توانند در **سیستم‌های در حال کار (Production Systems)** و بدون نیاز به راه‌اندازی مجدد یا ایجاد اختلال، برای عیب‌یابی و